Informe breve — Unidad 4: FNBC y Desnormalización Controlada en Food Store

Alumnos: equipo G · Cátedra: Base de Datos II (UTN) · Entrega: TP6

Repositorio con los tres artefactos: https://github.com/rayssa-ale/food-store/tree/main/TP6

Este PDF resuelve en un único archivo los dos ejercicios de la Unidad 4: la descomposición hasta FNBC de `control_lote_almacen` (Parte 1) y la desnormalización controlada del top 5 de categorías por día (Parte 2). Cada afirmación técnica va acompañada del SQL o de la salida de EXPLAIN ANALYZE que la demuestra.

1. Parte 1 — ControlLoteAlmacen

(a) Dependencias funcionales (notación formal)

Sobre el esquema R = {LoteID, DepositoID, ResponsableControlID} se desprenden, a partir del enunciado:

- FD1: {LoteID, DepositoID} → {ResponsableControlID}

"Para un lote y un depósito interviniente dados, el responsable de control queda unívocamente determinado."

- FD2: {ResponsableControlID} → {DepositoID}

"Cada responsable de control pertenece, como dato maestro de la dotación de personal, a un único depósito: no controla lotes coordinados desde depósitos distintos."

F+ contiene únicamente estas dos dependencias.

(b) Clausuras y claves candidatas

Clausuras relevantes:

- {ResponsableControlID}+ = {ResponsableControlID, DepositoID} (por FD2)
- {LoteID, DepositoID}+ = {LoteID, DepositoID, ResponsableControlID} (por FD1)
- {LoteID, ResponsableControlID}+ = {LoteID, ResponsableControlID, DepositoID} (aplica FD2 y luego FD1)

Claves candidatas (K):

- K1 = {LoteID, DepositoID} — superclave mínima
- K2 = {LoteID, ResponsableControlID} — superclave mínima

Atributos primos: LoteID, DepositoID, ResponsableControlID (todos pertenecen a alguna clave candidata). No existen atributos no primos.

(c) FNBC (definición formal)

Para toda DF X → Y no trivial en r, X debe ser superclave.

- FD1: determinante {LoteID, DepositoID} = K1 → es superclave → no viola.
- FD2: determinante {ResponsableControlID} → {ResponsableControlID}+ = {ResponsableControlID, DepositoID} ⊂ R → NO es superclave → **VIOLA FNBC**.

Por ello, `control_lote_almacen` no cumple FNBC. Sí cumple 3FN en este caso (no hay atributos no primos), lo cual es el caso canónico donde 3FN y FNBC se separan.

(d) Anomalías (instancia de ejemplo: (501,30,801), (502,30,801), (503,31,802))

1. **Inserción**: no se puede registrar que el responsable 803 pertenece al depósito 31 sin asignarlo a algún lote. Para darlo de alta se debe inventar un lote (ej. 504), lo cual crea una fila artificial sobre la relación de control.

2. **Borrado**: si se borra la única asignación del responsable 802 (lote 503), desaparece la única información que indica que 802 pertenece al depósito 31. Se pierde el dato maestro de dotación de personal al borrar una asignación.
3. **Actualización**: si el responsable 801 cambia de depósito (30 → 31), el cambio es múltiple: modificar (501,30,801) y (502,30,801) implica borrar e insertar filas (la PK incluye `deposito_id`) y, si se hace parcial (solo un lote), queda registrado en dos depósitos distintos, violando la regla "un único depósito".

Las tres anomalías están demostradas con SQL ejecutable dentro de SAVEPOINTs que se revierten (no alteran la base).

(e) Descomposición sin pérdida

Aplicando el algoritmo de descomposición para la DF violante X → Y, con X = {ResponsableControlID} e Y = {DepositoID}:

- r1 = X ∪ Y = {ResponsableControlID, DepositoID}

PK de r1 = {ResponsableControlID} → FNBC.

- r2 = R \ Y = {LoteID, ResponsableControlID}

PK de r2 = {LoteID, ResponsableControlID} → FNBC (sobre r2 no queda ninguna DF no trivial).

Tablas resultantes (ver `tp_fnbc_control_lote.sql`):

- `responsable_deposito(responsable_control_id PK FK→usuario, deposito_id FK→deposito, created_at)`
- `control_lote(lote_id FK→lote, responsable_control_id FK→usuario, created_at, PK(lote_id, responsable_control_id))`

Vista de compatibilidad (reunión natural sobre `responsable_control_id`):

```
CREATE VIEW control_lote_almacen_compat AS
SELECT cl.lote_id, rd.deposito_id, cl.responsable_control_id
  FROM control_lote cl
  JOIN responsable_deposito rd
    ON rd.responsable_control_id = cl.responsable_control_id;
```

Por qué no la variante "natural" (LoteID, DepositoID) + (ResponsableControlID, DepositoID). Esa pareja también queda en FNBC, pero se reúne por `DepositoID`, que **no** es clave de ninguna de las dos relaciones: un depósito tiene varios responsables y un depósito interviene en varios lotes, así que la reunión multiplica filas. Con dos responsables en el depósito 30, el lote 501 de ese depósito aparecería controlado por ambos. `tp_fnbc_control_lote.sql` lo demuestra con un contraejemplo ejecutable (tres tablas temporales y un conteo que da 4 filas donde el original tenía 2). Es el hallazgo **F-U4-4** del code review de la segunda entrega: la primera versión del script usaba esa variante y su `EXCEPT ALL` 0/0 pasaba únicamente porque la instancia de ejemplo tiene un responsable por depósito.

(f) Reunión sin pérdida — justificación (superclave sobre atributo común)

El atributo común de la reunión es `ResponsableControlID`. Teorema: la reunión natural de r1 y r2 sobre r1∩r2 es sin pérdida si y solo si r1∩r2 es una superclave de al menos una de las dos relaciones.

- `ResponsableControlID` es la clave primaria de `responsable_deposito` (r1) → **es superclave de r1**: por FD2 determina todo r1.
- Por consiguiente, `responsable_deposito` ■ `control_lote` es **sin pérdida**: no puede multiplicar filas (cada responsable aparece una sola vez en r1) ni perderlas (cada par lote–responsable de r2 recupera su depósito en r1).

Verificación empírica: `EXCEPT ALL` en ambas direcciones devuelve **0 filas**; cardinalidad idéntica (3 = 3) y firma (suma de claves) idéntica entre original y vista. El script agrega la prueba inyectiva por `(lote_id, deposito_id)`, que además funciona como detector de violaciones de FD1.

Costo declarado: la FNBC no preserva FD1. En `control_lote` ya no está `deposito_id`, así que `{LoteID, DepositoID} → ResponsableControlID` deja de ser una restricción declarativa: nada impide por esquema que un mismo lote sea controlado por dos responsables del mismo depósito. Es el trade-off clásico de FNBC frente a la síntesis 3FN (que sí preserva dependencias). Se declara explícitamente, se detecta con la consulta inyectiva sobre la vista y, si el negocio la exige como restricción dura, se resuelve con un disparador o volviendo a la síntesis 3FN.

2. Parte 2 — Consulta con desnormalización controlada

(a) EXPLAIN ANALYZE (línea base)

Consulta adaptada al esquema real (TIMESTAMPTZ vs DATE; baja lógica por ENUM):

```
SELECT c.nombre AS categoria,
       SUM(d.cantidad * d.precio_unitario) AS total_vendido
FROM detalle_pedido d
JOIN pedido pe ON pe.id = d.pedido_id
JOIN producto p ON p.id = d.producto_id
JOIN categoria c ON c.id = p.categoria_id
WHERE pe.fecha >= CURRENT_DATE
  AND pe.fecha < CURRENT_DATE + INTERVAL '1 day'
  AND pe.estado <> 'CANCELADO'
GROUP BY c.nombre
ORDER BY total_vendido DESC
LIMIT 5;
```

Nota sobre el predicado de fecha: `ped.fecha` es TIMESTAMPTZ y `CURRENT_DATE` es DATE; la igualdad literal `fecha = CURRENT_DATE` resuelve contra la medianoche local exacta y, sobre el día más cargado de la base (2026-09-22, 12.269 pedidos), devuelve **0 filas** en lugar de 12.269. Por eso se usa el predicado de rango `fecha >= CURRENT_DATE AND fecha < CURRENT_DATE + 1 day`, equivalente a `fecha::date = CURRENT_DATE` y sargable (usa `ix_pedido_fecha_estado`).

Medición sobre 15.000 pedidos / 37.500 líneas del día (base food_store_tp6):

- **Frío**: Execution Time = 233,234 ms
- **Caliente (mediana de 5 corridas)**: ≈ 138 ms (130,268 / 137,602 / 138,280 / 140,289 / 154,845)

Nodo dominante: `Parallel Seq Scan on public.detalle_pedido` (183.176 filas por worker × 3 = ~549.529 filas leídas) para responder sobre ~31.372 líneas. Lee ~17,5× más datos de los necesarios. Buffers: shared hit=1724 read=3280.

Plan completo (salida cruda del motor, sin editar):

```
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------

 Limit (cost=12680.76..12680.77 rows=5 width=210) (actual time=200.230..231.863 rows=3.00 loops=1)

   Output: c.nombre, (sum(((d.cantidad)::numeric * d.precio_unitario)))

   Buffers: shared hit=1724 read=3280

   -> Sort (cost=12680.76..12681.03 rows=110 width=210) (actual time=200.224..231.856 rows=3.00 loops=1)

         Output: c.nombre, (sum(((d.cantidad)::numeric * d.precio_unitario)))

         Sort Key: (sum(((d.cantidad)::numeric * d.precio_unitario))) DESC

         Sort Method: quicksort Memory: 25kB

         Buffers: shared hit=1724 read=3280

         -> Finalize GroupAggregate (cost=12644.83..12678.93 rows=110 width=210) (actual time=200.100..231.747 rows=3.00
loops=1)

               Output: c.nombre, sum(((d.cantidad)::numeric * d.precio_unitario))

               Group Key: c.nombre

               Buffers: shared hit=1721 read=3280

               -> Gather Merge (cost=12644.83..12675.58 rows=264 width=210) (actual time=200.080..231.719 rows=9.00 loops=1)

                     Output: c.nombre, (PARTIAL sum(((d.cantidad)::numeric * d.precio_unitario)))
```

```
                    Workers Planned: 2
                    Workers Launched: 2
                    Buffers: shared hit=1721 read=3280
                    -> Sort (cost=11644.80..11645.08 rows=110 width=210) (actual time=144.406..144.412 rows=3.00 loops=3)
                          Output: c.nombre, (PARTIAL sum(((d.cantidad)::numeric * d.precio_unitario)))
                          Sort Key: c.nombre
                          Sort Method: quicksort Memory: 25kB
                          Buffers: shared hit=1721 read=3280
                          Worker 0: actual time=112.012..112.016 rows=3.00 loops=1
                            Sort Method: quicksort Memory: 25kB
                            Buffers: shared hit=453 read=727
                          Worker 1: actual time=122.717..122.724 rows=3.00 loops=1
                            Sort Method: quicksort Memory: 25kB
                            Buffers: shared hit=408 read=1052
                          -> Partial HashAggregate (cost=11639.70..11641.08 rows=110 width=210) (actual
time=144.297..144.308 rows=3.00 loops=3)
                               Output: c.nombre, PARTIAL sum(((d.cantidad)::numeric * d.precio_unitario))
                               Group Key: c.nombre
                               Batches: 1 Memory Usage: 32kB
                               Buffers: shared hit=1705 read=3280
                               Worker 0: actual time=111.840..111.850 rows=3.00 loops=1
                                 Batches: 1 Memory Usage: 32kB
                                 Buffers: shared hit=445 read=727
                               Worker 1: actual time=122.611..122.623 rows=3.00 loops=1
                                 Batches: 1 Memory Usage: 32kB
                                 Buffers: shared hit=400 read=1052
                               -> Hash Join (cost=3999.30..11529.19 rows=11051 width=187) (actual time=98.021..131.896
rows=10453.00 loops=3)
                                    Output: c.nombre, d.cantidad, d.precio_unitario
                                    Inner Unique: true
                                    Hash Cond: (p.categoria_id = c.id)
                                    Buffers: shared hit=1705 read=3280
                                    Worker 0: actual time=66.105..100.314 rows=9979.00 loops=1
                                      Buffers: shared hit=445 read=727
                                    Worker 1: actual time=76.790..109.526 rows=10572.00 loops=1
                                      Buffers: shared hit=400 read=1052
                                    -> Parallel Hash Join (cost=3986.82..11486.59 rows=11051 width=17) (actual
time=96.456..124.542 rows=10453.00 loops=3)
                                         Output: d.cantidad, d.precio_unitario, p.categoria_id
                                         Inner Unique: true
                                         Hash Cond: (d.producto_id = p.id)
                                         Buffers: shared hit=1703 read=3279
                                         Worker 0: actual time=63.923..93.195 rows=9979.00 loops=1
                                           Buffers: shared hit=444 read=727
                                         Worker 1: actual time=75.954..103.439 rows=10572.00 loops=1
                                           Buffers: shared hit=399 read=1052
                                         -> Parallel Hash Join (cost=2491.69..9962.46 rows=11051 width=17) (actual
time=83.434..98.615 rows=10453.00 loops=3)
```

```
                                                Output: d.cantidad, d.precio_unitario, d.producto_id

                                                Inner Unique: true

                                                Hash Cond: (d.pedido_id = pe.id)

                                                Buffers: shared hit=1504 read=3279

                                                Worker 0: actual time=63.834..78.312 rows=9979.00 loops=1

                                                  Buffers: shared hit=444 read=727

                                                Worker 1: actual time=75.827..92.659 rows=10572.00 loops=1

                                                  Buffers: shared hit=399 read=1052

                                                -> Parallel Seq Scan on public.detalle_pedido d (cost=0.00..6869.70
rows=228970 width=25) (actual time=0.959..34.260 rows=183176.33 loops=3)

                                                      Output: d.id, d.pedido_id, d.producto_id, d.cantidad,
d.precio_unitario

                                                      Buffers: shared hit=1301 read=3279

                                                      Worker 0: actual time=1.826..30.262 rows=140449.00 loops=1

                                                        Buffers: shared hit=444 read=727

                                                      Worker 1: actual time=0.808..30.027 rows=174120.00 loops=1

                                                        Buffers: shared hit=399 read=1052

                                                -> Parallel Hash (cost=2411.13..2411.13 rows=6445 width=8) (actual
time=3.536..3.537 rows=4176.33 loops=3)

                                                      Output: pe.id

                                                      Buckets: 16384 Batches: 1 Memory Usage: 640kB

                                                      Buffers: shared hit=203

                                                      Worker 0: actual time=0.000..0.001 rows=0.00 loops=1

                                                      Worker 1: actual time=0.000..0.001 rows=0.00 loops=1

                                                      -> Parallel Bitmap Heap Scan on public.pedido pe
(cost=328.68..2411.13 rows=6445 width=8) (actual time=1.902..6.705 rows=12529.00 loops=1)

                                                            Output: pe.id

                                                            Recheck Cond: ((pe.fecha >= CURRENT_DATE) AND (pe.fecha <
(CURRENT_DATE + '1 day'::interval)))

                                                            Filter: (pe.estado <> 'CANCELADO'::estado_pedido)

                                                            Rows Removed by Filter: 2471

                                                            Heap Blocks: exact=126

                                                            Buffers: shared hit=203

                                                            -> Bitmap Index Scan on ix_pedido_fecha_estado
(cost=0.00..325.94 rows=12951 width=0) (actual time=1.767..1.767 rows=15000.00 loops=1)

                                                                  Index Cond: ((pe.fecha >= CURRENT_DATE) AND (pe.fecha <
(CURRENT_DATE + '1 day'::interval)))

                                                                  Index Searches: 1

                                                                  Buffers: shared hit=77

                                          -> Parallel Hash (cost=1234.68..1234.68 rows=20836 width=16) (actual
time=12.837..12.838 rows=16668.67 loops=3)

                                                Output: p.id, p.categoria_id

                                                Buckets: 65536 Batches: 1 Memory Usage: 2880kB

                                                Buffers: shared hit=199

                                                Worker 0: actual time=0.061..0.062 rows=0.00 loops=1

                                                Worker 1: actual time=0.027..0.028 rows=0.00 loops=1

                                                -> Parallel Index Only Scan using ix_producto_cat_cover on public.producto
p (cost=0.29..1234.68 rows=20836 width=16) (actual time=0.027..12.841 rows=50006.00 loops=1)

                                                      Output: p.id, p.categoria_id

                                                      Heap Fetches: 70

                                                      Index Searches: 1
```

```
                                                    Buffers: shared hit=199
                                    -> Hash (cost=11.10..11.10 rows=110 width=186) (actual time=1.542..1.542 rows=3.00
loops=3)
                                          Output: c.nombre, c.id
                                          Buckets: 1024 Batches: 1 Memory Usage: 9kB
                                          Buffers: shared hit=2 read=1
                                          Worker 0: actual time=2.153..2.153 rows=3.00 loops=1
                                            Buffers: shared hit=1
                                          Worker 1: actual time=0.819..0.820 rows=3.00 loops=1
                                            Buffers: shared hit=1
                                          -> Seq Scan on public.categoria c (cost=0.00..11.10 rows=110 width=186) (actual
time=1.514..1.516 rows=3.00 loops=3)
                                                Output: c.nombre, c.id
                                                Buffers: shared hit=2 read=1
                                                Worker 0: actual time=2.105..2.108 rows=3.00 loops=1
                                                  Buffers: shared hit=1
                                                Worker 1: actual time=0.805..0.806 rows=3.00 loops=1
                                                  Buffers: shared hit=1
 Planning:
   Buffers: shared hit=553 read=2
 Planning Time: 18.380 ms
 Execution Time: 233.234 ms
(120 filas)
```

El plan completo, sin recortar, es el de arriba: va desde el nodo Limit hasta Execution Time.

(b) Justificación del patrón elegido

Se eligió **estructura agregada precalculada mantenida con disparadores** (`fact_venta_categoria_dia` + triggers AFTER sobre `detalle_pedido`, `pedido`, `producto`, `categoria`), por sobre una vista materializada.

- **¿Qué evidencia medida motiva la decisión?** La consulta de línea base está dominada por `Parallel Seq Scan` sobre `detalle_pedido`: lee ~549.529 filas (85 MB de tabla) para filtrar un día y agrupar por categoría (~31.372 líneas útiles). El costo NO está en el JOIN final sino en recorrer la tabla de líneas completa. Un cubo con grano (fecha, categoría) reduce la lectura del día corriente a **1 bloque y 3 filas**, atacando exactamente el nodo dominante del plan medido.
- **¿Qué mecanismo garantiza que el dato redundante no se desincronice?** Dos caminos, ambos dentro de la misma transacción que escribe la fuente (por eso o escriben ambos o no escribe ninguno):
- *Camino caliente* (INSERT/DELETE de `detalle_pedido`): ajuste incremental del cubo con delta atómico (`ajustar_fact_venta`), sumando sólo si el pedido **no está CANCELADO** — el mismo criterio del `WHERE` original.
- *Camino frío* (cambios de alcance ancho: mover la fecha de un pedido, cruzar la frontera CANCELADO, recategorizar un producto): `recalcular_fact_venta_dia(fecha)` **reconstruye el día completo** con la misma agregación que usa el auditor; es correcto por construcción y no depende de propagar impactos a mano.
- El nombre de categoría (dato copiado) se propaga con trigger `AFTER UPDATE OF nombre ON categoria`, y el backfill usa la misma agregación.
- La **auditoría (5.2e)** recalcula desde la fuente de verdad y compara en **ambas direcciones** los tres datos redundantes (total, líneas y nombre). Resultado vacío sobre la base migrada; y se demuestra que tiene dientes rompiendo el dato a propósito dentro de un ROLLBACK.

- **¿Es reversible sin pérdida de información?** Sí. `fact_venta_categoria_dia` es **100 % derivable** de `pedido`/`detalle_pedido`/`producto`/`categoria`. Se elimina con `DROP TABLE` y se reconstruye ejecutando el backfill incluido. Ninguna fila de la fuente de verdad se modifica.

Por qué no una vista materializada: el panel requiere tiempo real con actualización frecuente. Una MV exige `REFRESH` (completo o `CONCURRENTLY`), un proceso por lote y no reactivo a cada línea; entre refrescos el dato queda viejo y hay ventana de desincronización. Los disparadores mantienen el cubo al día por fila y con garantías ACID. TP5 ya midió MV para otro grano (mes/forma de pago) y con otro propósito.

(c) Implementación

Ver `tp_desnormalizacion_top_categorias.sql`:

- Tabla: `fact_venta_categoria_dia(fecha DATE, categoria_id BIGINT, categoria_nombre VARCHAR(80), total_vendido NUMERIC(14,2), lineas INTEGER, actualizado_en TIMESTAMPTZ, PK(fecha,categoria_id), FK→categoria, CHECK no negativos)` — grano = clave natural, lo que habilita upserts incrementales.
- Funciones: `ajustar_fact_venta`, `recalcular_fact_venta_dia`, y las funciones de trigger (ins/del/upd de detalle; upd de pedido; upd de producto; upd de categoría).
- Backfill inicial con la misma agregación del auditor.
- Pruebas de sincronización (INSERT y cancelación) dentro de transacciones revertidas.

(d) Comparación antes/después (EXPLAIN ANALYZE)

Consulta sobre la estructura desnormalizada:

```
SELECT fv.categoria_nombre AS categoria, fv.total_vendido
  FROM fact_venta_categoria_dia fv
 WHERE fv.fecha = CURRENT_DATE
 ORDER BY fv.total_vendido DESC
 LIMIT 5;
```

Después (mediana de 5 corridas): Execution Time ≈ **0,118 ms** (0,104 / 0,109 / 0,118 / 0,175 / 0,179 ms). Plan: `Bitmap Index Scan` sobre `pk_fact_venta_categoria_dia` → `Bitmap Heap Scan` (Heap Blocks: 1) → `Sort` por `total_vendido DESC`. Buffers: shared hit=6.

Plan completo (salida cruda del motor):

```
                                                
```

```
                    Index Searches: 1
                    Buffers: shared hit=2
 Planning:
   Buffers: shared hit=121
 Planning Time: 2.789 ms
 Execution Time: 0.155 ms
(21 filas)
```

	Antes (normalizado, 4 tablas)	Después (e
	Parallel Seq Scan sobre detalle_pedido	**Bitmap Inde**
	~549.529 (detalle_pedido) para ~31.372 líneas útiles	~7 (índice) –
	shared hit=1724 read=3280	shared hit=6
	233,234 ms	**0,155 ms**
	138,0 ms	**0,118 ms**

Ganancia ≈ **×1.170** (138 ms → 0,118 ms). El salto es cualitativo: de leer medio millón de filas a leer 3.

(e) Script de auditoría (resultado vacío sobre la base migrada)

La auditoría (5.2e) compara `(fecha, categoria_id, categoria_nombre, total_vendido, lineas)` entre la verdad (4 tablas, `estado <> 'CANCELADO'`) y lo almacenado, en **ambas direcciones** (`cubo_faltante` / `distinto`). Sobre la base migrada devuelve **0 filas**. Se incluye además una prueba que rompe un monto a propósito (dentro de ROLLBACK) para confirmar que la auditoría detecta la desincronización.

3. Artefactos entregados

1. `tp_fnbc_control_lote.sql` — Parte 1: DDLs, instancia de ejemplo, demostración de las 3 anomalías (con SAVEPOINTs), descomposición, vista de compatibilidad, migración verificada con EXCEPT ALL y firma de equivalencia, contraejemplo de la variante con pérdida, justificación teórica de la reunión sin pérdida.
2. `tp_desnormalizacion_top_categorias.sql` — Parte 2: estructura desnormalizada, mecanismo de sincronización (disparadores), backfill, equivalencia EXCEPT ALL entre antes/después, auditoría con doble dirección, prueba de detección (deliberate break) y pruebas de sincronización (todas revertidas con ROLLBACK), sección de reversibilidad.
3. Este informe en PDF: `TP6/TP6_Food_Store.pdf`.

Reproducir las mediciones. La Parte 2 necesita que el día corriente tenga pedidos; `tp_desnormalizacion_top_categorias.sql` lo explica en su encabezado y `TP6/carga_dia_actual.sql` lo siembra. Ambos scripts son idempotentes (`DROP ... IF EXISTS` + `CREATE`) y todas sus pruebas corren en transacciones revertidas: la base queda con el esquema final y sin datos de prueba.

Repositorio: https://github.com/rayssa-ale/food-store/tree/main/TP6 (rama `main`, carpeta `TP6/`).

Ejecución verificada sobre `food_store_tp6` (PostgreSQL 18 local): la Parte 2 (mediciones y planes) se mantiene tal como se ejecutó y documentó. La Parte 1 fue **corregida** en la segunda entrega (F-U4-4, §4): su lógica fue validada por simulación, pero debe **reejecutarse** para refrescar la salida verificada (`EXCEPT ALL` 0/0, cardinalidad, firma y contraejemplo). Toda afirmación técnica está acompañada del SQL o de la salida que la demuestra.

4. Corrección posterior incorporada en la segunda entrega (F-U4-4)

El code review de la segunda entrega detectó que la primera versión de la Parte 1 descomponía en `responsable_deposito(Responsable, Deposito)` + `control_lote(Lote, Deposito)` y reunía por `DepositoID`. Esa reunión es **con pérdida**: `{DepositoID}+ = {DepositoID}` no es superclave de ninguna de las dos relaciones, y su `EXCEPT ALL` 0/0 pasaba únicamente porque la instancia de ejemplo tiene un responsable por depósito. La corrección —validada por el equipo— cambia `control_lote` a `(lote_id, responsable_control_id)`, que se

reúne por `ResponsableControlID` (clave de `responsable_deposito`), agrega el contraejemplo que exhibe la pérdida de la variante descartada y declara el costo de preservación de FD1 (secciones (e) y (f) de este informe).

Las secciones (a)–(d) y toda la Parte 2 no cambiaron. Tras la corrección, la Parte 1 debe **reejecutarse** sobre `food_store_tp6` para refrescar la salida verificada (`EXCEPT ALL` 0/0, cardinalidad y firma):

```
& 'C:\Program Files\PostgreSQL\18\bin\psql.exe' -U postgres -h localhost `
    -d food_store_tp6 -v ON_ERROR_STOP=1 -f TP6/tp_fnbc_control_lote.sql
```

El detalle del hallazgo, con severidad y decisión, está en `SEGUNDA_ENTREGA/code_review_segunda_entrega.md` (F-U4-4); la interacción con la IA y la decisión humana, en `BITACORA.md` (U4-4) y `docs/DUIA.md`.